

EXPERIMENT 25

Title: Write a Prolog Program to Implement Monkey Banana Problem

Aim

To develop a Prolog program to solve the Monkey Banana Problem using logical reasoning and rules, where a monkey obtains a banana by moving to the required position, moving a box under the banana, climbing the box, and grasping the banana.

Objective

- To understand problem-solving using Prolog.
- To represent states and actions using predicates.
- To implement logical reasoning using facts and rules.
- To understand state-space representation.
- To solve the Monkey Banana Problem using Prolog.

Theory

The Monkey Banana Problem is a classical Artificial Intelligence problem used to demonstrate problem solving and state-space search.

In this problem, a monkey is in a room containing a banana hanging from the ceiling and a box. The monkey cannot directly reach the banana. The monkey must perform a sequence of actions:

- Move to the location of the box.
- Push the box to the position below the banana.
- Climb onto the box.
- Grasp the banana.

Prolog can represent these actions as logical rules and determine whether the monkey can obtain the banana.

Algorithm

Step 1: Start the program.

Step 2: Represent the initial position of the monkey, box, and banana.

Step 3: Check whether the monkey is at the same location as the box.

Step 4: If necessary, move the monkey to the box.

Step 5: Push the box to the location below the banana.

Step 6: Climb onto the box.

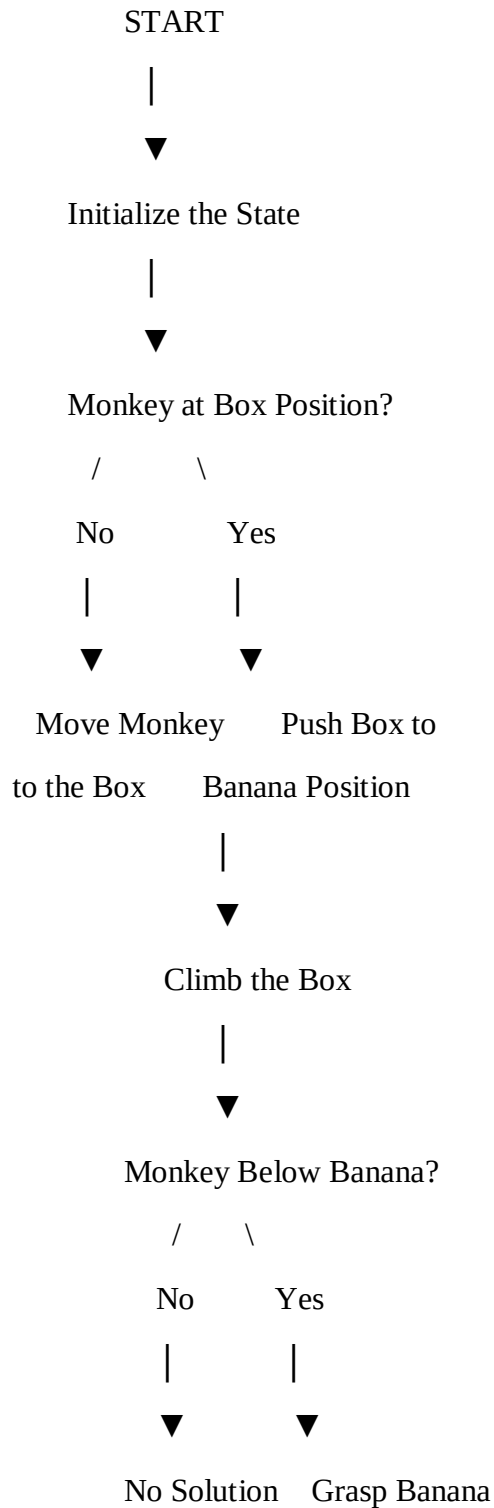
Step 7: Check whether the monkey is below the banana.

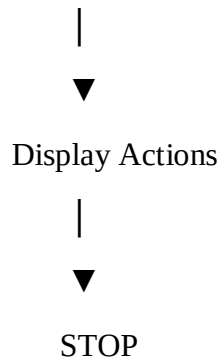
Step 8: If the monkey is on the box below the banana, grasp the banana.

Step 9: Display the sequence of actions.

Step 10: Stop the program.

Flowchart





Prolog Program

% Monkey Banana Problem

% Initial state:

% Monkey is at door, box is at window,

% banana is hanging at the middle.

% Move monkey to the box move_monkey(door,
window, window).

% Push box from window to middle
push_box(window, middle, middle).

% Climb onto the box climb_box(middle,
middle, on_box).

% Grasp banana grasp_banana(on_box,
middle, middle).

% Rule to solve the Monkey Banana Problem

can_get_banana :- move_monkey(door,
window, window), push_box(window,
middle, middle), climb_box(middle,

```
middle, on_box), grasp_banana(on_box,
middle, middle), write('Monkey can get
the banana. '), nl.
```

Query

```
?- can_get_banana.
```

Sample Output

```
Monkey can get the banana.
```

```
true.
```

Query for Individual Actions

The movement of the monkey can also be queried:

```
?- move_monkey(door, window, X).
```

Sample Output

```
X = window.
```

The box movement can be queried using:

```
?- push_box(window, middle, X).
```

Sample Output X

```
= middle.
```

Explanation

The Monkey Banana Problem is represented using a sequence of predicates.

The predicate:

`move_monkey(door, window, window).` represents the movement of the monkey from the door to the location of the box.

The predicate:

`push_box(window, middle, middle).` represents pushing the box from the window to the position below the banana.

The predicate:

`climb_box(middle, middle, on_box).` represents the monkey climbing onto the box.

Finally:

grasp_banana(on_box, middle, middle). represents the monkey grasping the banana after reaching the required position.

The can_get_banana rule combines all these actions in the correct sequence. Prolog verifies each condition and concludes that the monkey can obtain the banana.

Advantages

- Demonstrates state-space problem solving.
- Uses logical rules to represent actions.
- Simple representation of an AI planning problem.
- Demonstrates sequential reasoning.
- Easy to modify for different initial states.

Applications

- Artificial Intelligence planning.
- Automated reasoning.
- Robotics.
- Game problem solving.
- State-space search.
- Intelligent planning systems.

Result

The Prolog program was successfully implemented to solve the Monkey Banana Problem. The program generated the required sequence of actions and successfully determined that the monkey can obtain the banana.

Conclusion

Thus, the Monkey Banana Problem was successfully implemented using Prolog. The experiment demonstrates how logical rules and sequential actions can be used to solve a classical Artificial Intelligence problem through state representation and reasoning.

```

7 Users > varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPS > EXP1.py > monkey_banana_problem
1  # MONKEY BANANA PROBLEM
2
3  # Initial state
4  monkey_position = "door"
5  box_position = "window"
6  monkey_on_box = False
7  banana_obtained = False
8
9
10 def monkey_banana_problem():
11     global monkey_position, box_position
12     global monkey_on_box, banana_obtained
13
14     print("Initial State:")
15     print("Monkey is at:", monkey_position)
16     print("Box is at:", box_position)
17     print("Bananas are hanging from the ceiling.")
18     print()
19
20     # Step 1: Monkey walks to the box
21     print("1. Monkey walks from door to window.")
22     monkey_position = "window"
23
24     # Step 2: Monkey pushes the box under the bananas
25     print("2. Monkey pushes the box under the bananas.")
26     box_position = "under_banana"
27
28     # Step 3: Monkey climbs onto the box
29     print("3. Monkey climbs onto the box.")
30     monkey_on_box = True
31
32     # Step 4: Monkey grabs the bananas
33     if monkey_on_box and box_position == "under_banana":
34         print("4. Monkey grabs the bananas.")
35         banana_obtained = True
36
37     # Final state
38     if banana_obtained:
39         print("\nSuccess! Monkey has obtained the bananas.")
40     else:
41         print("\nMonkey could not obtain the bananas.")
42
43
44 # Run the program
45 monkey_banana_problem()

```

```

PS C:\Users\Varshini M> python "C:\Users\Varshini M\OneDrive\Des
Initial State:
Monkey is at: door
Box is at: window
Bananas are hanging from the ceiling.

1. Monkey walks from door to window.
2. Monkey pushes the box under the bananas.
3. Monkey climbs onto the box.
4. Monkey grabs the bananas.

Success! Monkey has obtained the bananas.

```